LoDaStock

Documentation Technique

Projet de Stage de Rayane Watteeuw

Loiselet & Daigremont

Table des matières

1. Présentation du projet 3
1.1 Contexte 3
1.2 Objectifs 3
1.3 Interlocuteurs 3
2. Spécifications fonctionnelles 3
2.1 Module : Gestion du Catalogue et des Stocks 3
2.2 Module : Gestion des Tiers (Fournisseurs) 4
2.3 Module : Workflow d'Approvisionnement (Commandes) 4
2.4 Module : Suivi Administratif et Facturation 4
2.5 Système de Notifications et Tableau de Bord 4
3. Spécifications techniques 4
3.1 Architecture Logicielle 4
3.2 Outils Technologiques 5
4. Acteurs et droits d'accès 5
5. Livrables et résultats attendus 5
6. Modélisation du système (UML): 6
6.1 Diagramme de Cas d'Utilisation 6
6.2 Diagrammes d'Activité 6
6.2.1 Processus de Commande et choix du canal 7
6.2.2 Algorithme d'Alerte de Stock 8
7. Conception de la base de données : 9
7.1 Modèle MCD : 9
7.2 Modèle MLD : 10
7.2.1 MLD Textuel : 11
7.3 Modèle MPD : 12
7.3.1 Description du MPD : 15
7.4 Automatisation (Triggers SQL) 15
8. Modèle MVC 17
8.1 Modèle (Model) 17
8.2 Contrôleur (Controller) 18
8.3 Vue (View) 19
9. Problème rencontré 20
9.1 Gestion des envois d'emails et Commandes Fournisseurs 20
9.2 Intégrité des données et politique de suppression 21
9.3 Système d'alerte de stock : Alternative aux notifications par email 22
10. Gestion des erreurs 22
10.1 Côté Serveur (Backend) : Centralisation et Sécurité de la donnée 23
10.2 Côté Client (Frontend) : Retours visuels dynamiques 23
11. Sécurité et Audit des Vulnérabilités (OWASP ZAP) 23
11.1 Renforcement des politiques de sécurité (Mitigation) 24
11.2 Alertes informatives et Faux positifs 26
12. Conteneurisation et Déploiement 26
12.1 Les avantages de la solution Docker 26
12.2 Architecture mise en place 26
13. Axes d'amélioration et Évolutions futures 27

1. Présentation du projet

1.1 Contexte

Dans le cadre de ses activités de maintenance de copropriétés, l'entreprise doit gérer un flux constant de matériels et consommables. Actuellement, le suivi des stocks et les interactions avec les fournisseurs manquent d'optimisation, entraînant des risques de rupture de stock ou de sur-stockage.

1.2 Objectifs

L'objectif est de développer une solution web centralisée ("Intranet") permettant de digitaliser l'ensemble de la chaîne logistique interne. Les buts stratégiques sont :

- **Fiabiliser** l'état des stocks en temps réel.
- **Fluidifier** le processus de réapprovisionnement auprès des fournisseurs.
- **Automatiser** les tâches administratives (mises à jour de stock, alertes).
- **Assurer la traçabilité** des mouvements et des coûts.

2. Spécifications fonctionnelles

2.1 Module : Gestion du Catalogue et des Stocks

Ce module constitue le cœur du système, garantissant l'unicité et la disponibilité des produits.

- **Administration du catalogue (CRUD) :** Ajout, modification et archivage des fiches produits.
- **Attributs gérés :** Libellé, Type (catégorisation), Prix unitaire indicatif, Référence interne.
- **Gestion des niveaux de stock :**
    - Visualisation immédiate des quantités disponibles.
    - Définition de **seuils d'alerte** personnalisables par produit (Stock minimum).
    - **Règle de gestion :** Unicité du stock par produit (Gestion centralisée, pas de multi-dépôts).

2.2 Module : Gestion des Tiers (Fournisseurs)

Centralisation des données partenaires pour faciliter la prise de commande.

- **Base de données fournisseurs :** Création de fiches complètes (Raison sociale, Ville, Email, Téléphone).
- **Fonctionnalités de contact :**
    - Intégration de liens "Mailto" pour l'envoi rapide d'emails.
    - Ajouter l’option de mail automatique.
- **Historique :** Accès rapide à l'ensemble des commandes passées chez un fournisseur spécifique.

2.3 Module : Workflow d'Approvisionnement (Commandes)

Gestion du cycle de vie complet d'une commande d'achat.

- **Création de commande :** Sélection du fournisseur et constitution du panier (Lignes de commande : Produit + Quantité).
- **Gestion des statuts :**
    - *BROUILLON :* Préparation de la commande.
    - *COMMANDÉE :* Validation et envoi au fournisseur.
    - *REÇUE :* Validation de la livraison physique.
    - *ANNULÉE :* Archivage sans impact stock.
- **Automatisation (Trigger) :** Lors du passage au statut **"REÇUE"**, le système incrémente automatiquement les quantités en stock sans intervention manuelle supplémentaire.

2.4 Module : Suivi Administratif et Facturation

- **Réconciliation :** Possibilité d'associer un numéro de facture ou un document numérisé à une commande réceptionnée.
- **Archivage :** Conservation de l'historique des factures pour les besoins comptables.

2.5 Système de Notifications et Tableau de Bord

Le système doit être proactif pour assister l'utilisateur.

- **Dashboard :** Vue synthétique présentant les indicateurs clés (KPI) et les actions urgentes.
- **Alertes automatisées :**
    - **Rupture :** Notification visuelle immédiate si quantité = 0.
    - **Seuil critique :** Notification si quantité < seuil défini, invitant au réapprovisionnement.

3. Spécifications techniques

3.1 Architecture Logicielle

L'application sera développée sous forme d'application web, accessible via un navigateur standard, ne nécessitant aucune installation sur les postes clients.

3.2 Outils Technologiques

Le choix des technologies privilégie la rapidité de développement, la robustesse et la maintenabilité.

- **Backend (Serveur) : PHP / Laravel**
    - Utilisation du framework Laravel pour sa structure MVC, sa sécurité native (protection CSRF/SQL Injection) et son puissant ORM (Eloquent).
- **Base de Données : SQLite**
    - Solution légère et portable, parfaitement adaptée à la volumétrie du projet et ne nécessitant pas de serveur de base de données lourd.
    - Implémentation de **Triggers SQL** pour l'automatisation des mouvements de stock.
- **Frontend (Interface) : Bootstrap & Alpine.js**
    - **Bootstrap 5 :** Pour une interface responsive (adaptée mobile/bureau) et professionnelle.

- **Alpine.js :** Pour gérer l'interactivité côté client (modales, mises à jour dynamiques) sans la lourdeur d'un framework type Vue/React.

- **Assistant coding: Gemini Pro & Copilote Pro**
  - **Gemini Pro :** Utilisé pour l'analyse des besoins et la rédaction de la documentation technique.
  - **Copilote Pro :** Employé comme assistant de développement pour accélérer l'implémentation du code et améliorer la productivité.

4. Acteurs et droits d'accès

L'accès à l'application est sécurisé par authentification (Login/Mot de passe).

- **Profil ADMINISTRATEUR :**
  - Accès total au système.
  - Gestion des utilisateurs (création de comptes employés).
  - Configuration des paramètres globaux.
  - Suppression des données.
- **Profil GESTIONNAIRE (User) :**
  - Gestion opérationnelle quotidienne.
  - Consultation des stocks, passage de commandes, validation des réceptions.
- **Profil SPECTATEUR (GUEST) :**
  - Peut uniquement voir les informations
  - Ne peut pas interagir avec les boutons de Modification de Création et de Suppression

5. Livrables et résultats attendus

À la livraison du projet, l'application devra permettre :

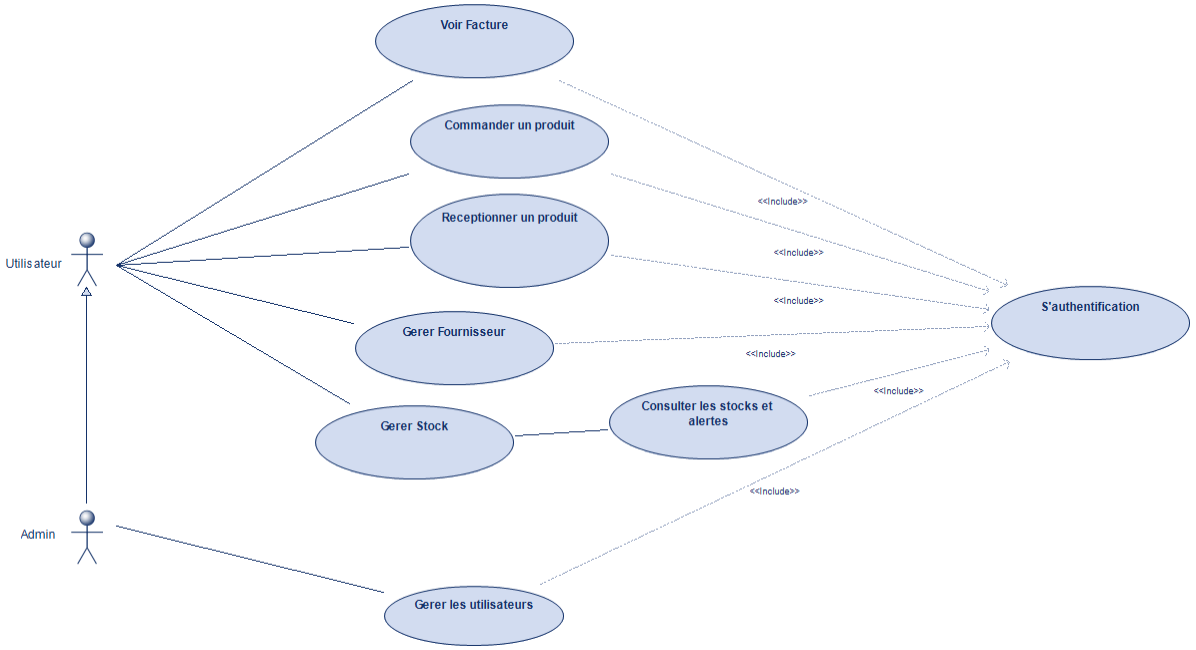
1. **Une réduction drastique des ruptures de stock** grâce aux alertes préventives.
2. **Un gain de temps administratif** estimé grâce à l'automatisation des entrées de stock.
3. **Une centralisation de l'information**, éliminant les fichiers Excel dispersés et les pertes d'informations fournisseurs.

6. Modélisation du système (UML):

Avant d'aborder la structure technique des données, il est essentiel de modéliser le comportement dynamique de l'application **LoDaStock**. Nous avons utilisé le langage UML (Unified Modeling Language) pour formaliser les interactions entre les acteurs et le système, ainsi que pour détailler les algorithmes complexes liés aux commandes et aux alertes.

6.1 Diagramme de Cas d'Utilisation

Ce diagramme offre une vue d'ensemble des fonctionnalités offertes par le système et définit les droits d'accès. Il permet de visualiser "qui fait quoi".



Analyse du diagramme :

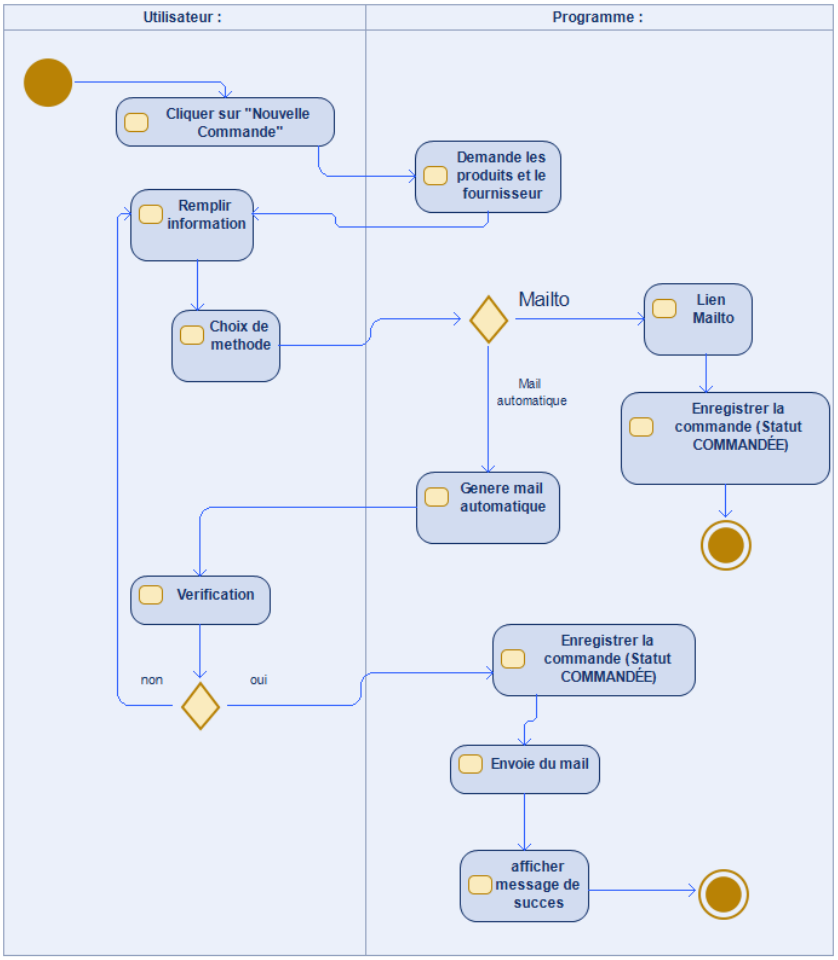
- **Les Acteurs :** Nous distinguons deux acteurs principaux. L'**Utilisateur** (gestionnaire standard) et l'**Administrateur**. Une relation d'héritage (généralisation) lie l'Administrateur à l'Utilisateur, signifiant que l'admin possède tous les droits de base (commander, réceptionner) en plus de ses droits spécifiques (gestion des comptes utilisateurs). De plus, le Guest ne peut que voir les données mais pas interagir avec.
- **Sécurisation :** L'ensemble des cas d'utilisation (hormis la connexion elle-même) nécessite une authentification préalable, garantissant la sécurité des actions.
- **Séparation des flux :** Les actions de "Commander un produit" et "Réceptionner une commande" sont distinctes. Cette séparation est cruciale car la réception est l'événement déclencheur de la mise à jour des stocks.

6.2 Diagrammes d'Activité

Les diagrammes d'activité permettent de détailler le déroulement chronologique des processus métier complexes.

6.2.1 Processus de Commande et choix du canal

Ce diagramme détaille l'algorithme de création d'une commande, en mettant l'accent sur la flexibilité du mode d'envoi (Email automatique ou Client de messagerie local).

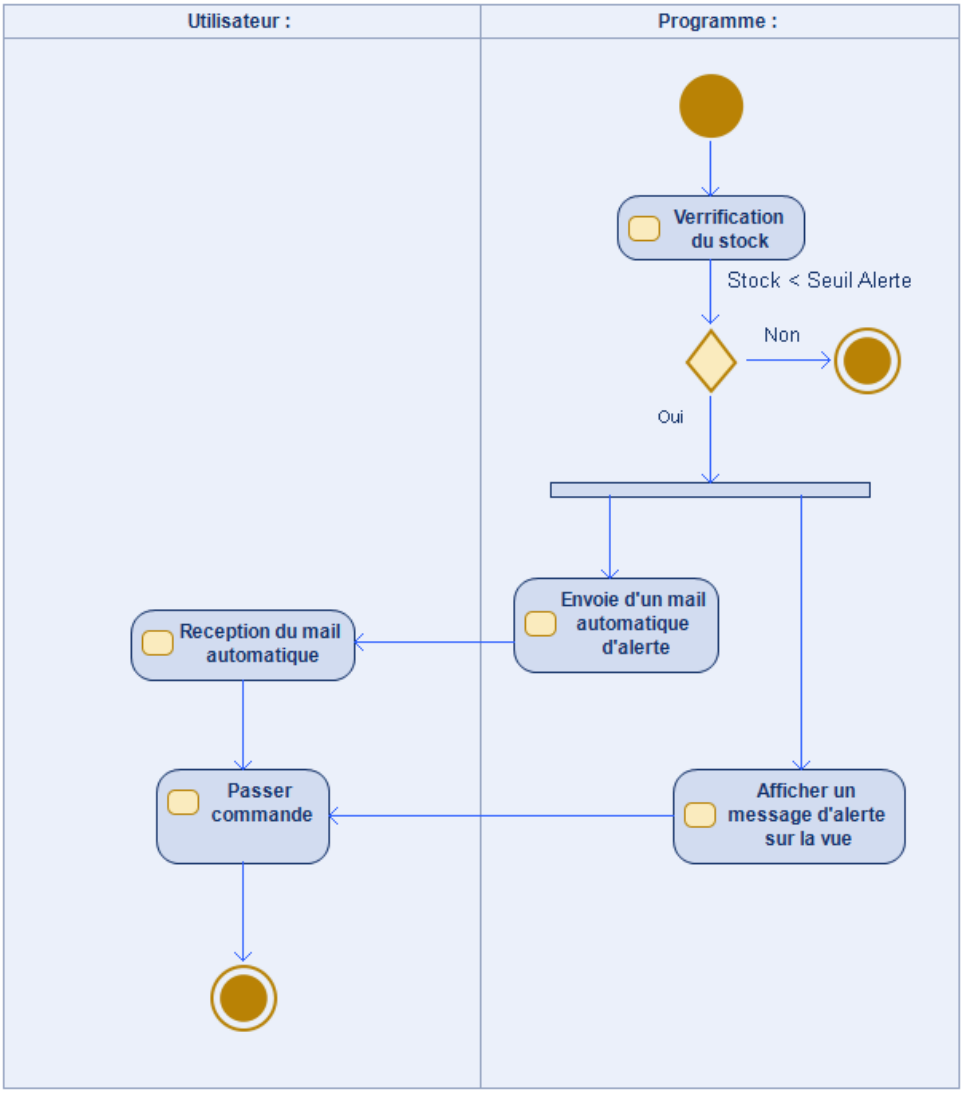


Description du flux : Le processus a été conçu pour garantir l'intégrité des données, quel que soit le mode d'envoi choisi par l'utilisateur.

1. **Saisie préalable :** L'utilisateur sélectionne le fournisseur et les produits **avant** de choisir le mode d'envoi. Cela permet au système de constituer l'objet "Commande".
2. **Enregistrement sécurisé :** Que l'utilisateur choisisse l'envoi automatique (via le serveur SMTP) ou manuel (via un lien `mailto:`), la commande est impérativement enregistrée en base de données avec le statut "COMMANDÉE".
3. **Finalisation :** Le stock n'est pas modifier à cette étape, il le sera uniquement lors de la réception future de cette commande.

6.2.2 Algorithme d'Alerte de Stock

Ce diagramme illustre l'automatisation de la surveillance des stocks, répondant à l'objectif de "prévention des ruptures" défini dans le cahier des charges.

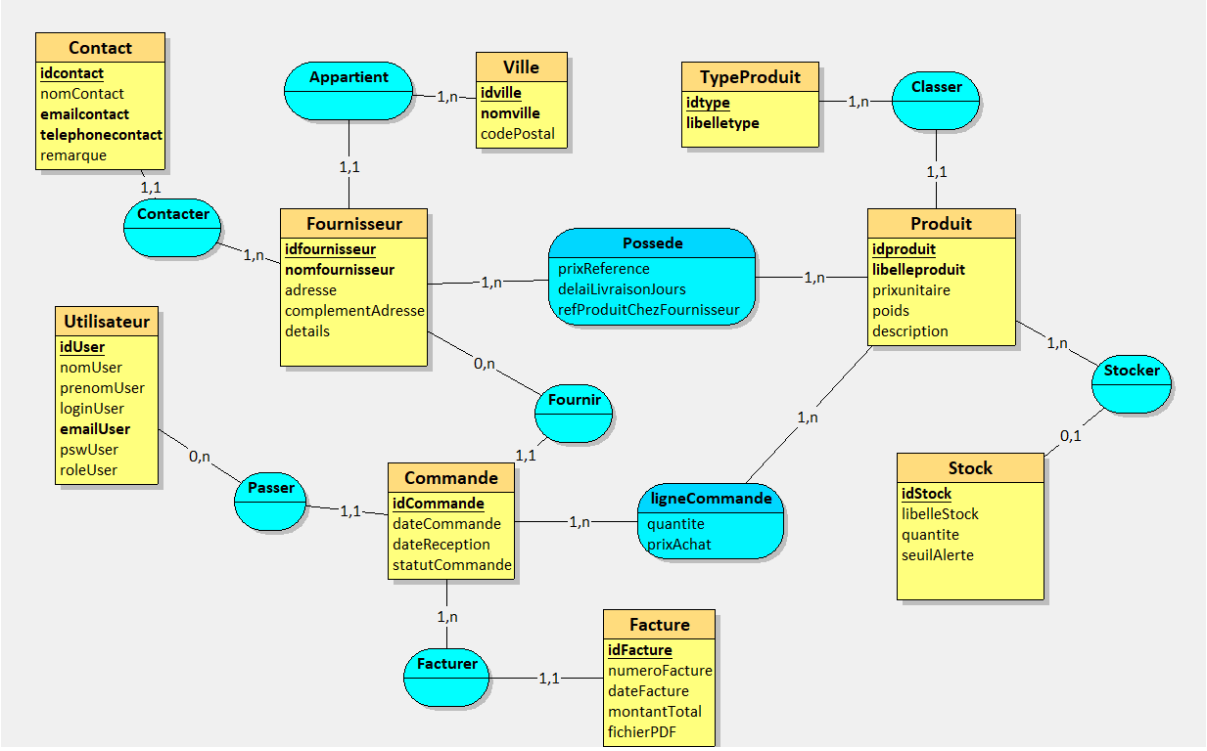


Description du flux :

1. **Déclenchement :** Le processus est initié automatiquement (par exemple après chaque mouvement de stock ou via une tâche planifiée exemple tous les vendredis).
2. **Condition de seuil :** Le système vérifie pour chaque produit si la quantité actuelle est inférieure au `seuilAlerte` défini et qu'il n'y a pas de commande en cours avec ce produit.
3. **Actions parallèles :** Si la condition est vérifiée, le système exécute deux actions simultanées (Fork) :
   - Envoi d'une notification par email au gestionnaire.
   - Mise à jour visuelle du tableau de bord (Dashboard).
4. **Résolution :** Ce flux incite l'utilisateur à lancer une procédure de réapprovisionnement.

7. Conception de la base de données :

7.1 Modèle MCD :



Ce schéma relationnel a été conçu pour structurer de manière exhaustive la chaîne logistique de l'entreprise, de la gestion des tiers jusqu'au stockage physique.

1. Gestion des Tiers et de la Localisation Nous avons défini une entité **Utilisateur**, regroupant les employés (gestionnaires et administrateurs) avec leurs identifiants de connexion sécurisés et leur rôle. Les commandes sont passées auprès de **Fournisseurs**. Chaque fournisseur est localisé géographiquement via une relation avec l'entité **Ville** (migration de la clé primaire `idville`). *Nouveauté majeure :* La structure a été affinée pour distinguer l'entreprise de ses interlocuteurs. La table Fournisseur stocke désormais l'adresse physique complète (indispensable pour la facturation), tandis qu'une nouvelle entité **Contact** permet d'enregistrer plusieurs personnes physiques (commercial, comptable) rattachées à un même fournisseur, avec leurs coordonnées directes.

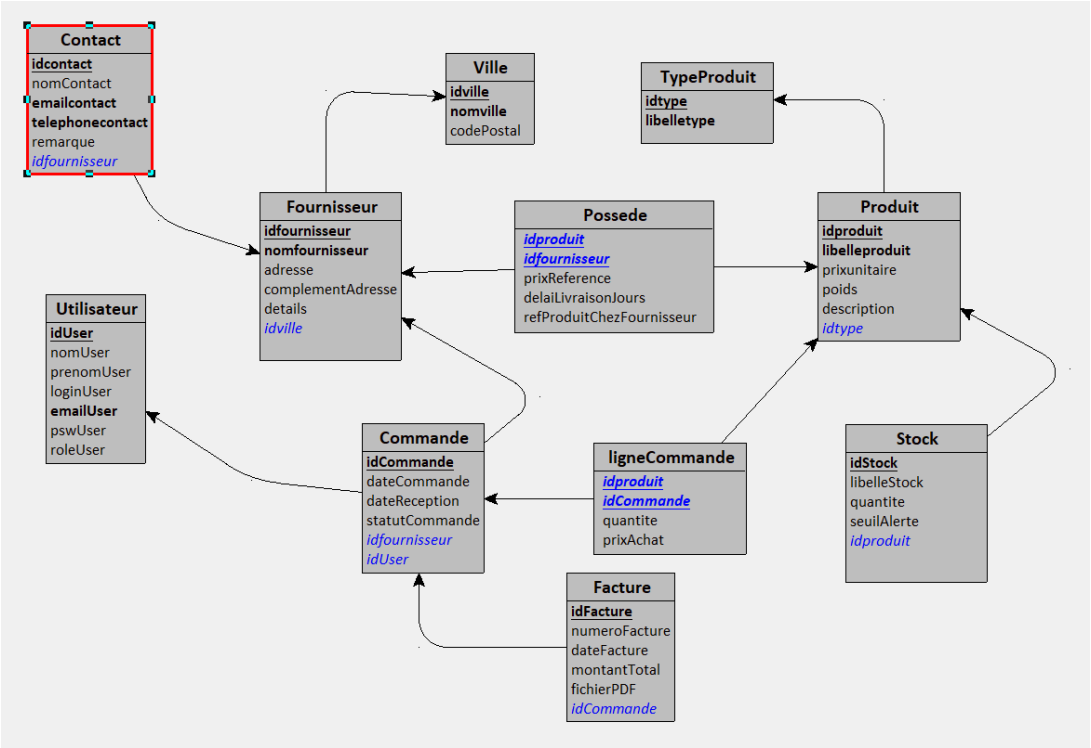
2. Catalogue Produit Au cœur du système se trouve le **Produit**, classé par **TypeProduit** (ex: Outillage, Consommable). La fiche produit a été enrichie pour inclure non seulement le prix unitaire et le libellé, mais également le **poids** et une **description** détaillée, facilitant la logistique. La table associative **Possede** (Catalogue Fournisseur) permet de référencer quel fournisseur vend quel produit, à quel prix négocié et avec quel délai de livraison.

3. Workflow de Commande et Facturation Les utilisateurs sont les acteurs déclencheurs des **Commandes**. Chaque commande est caractérisée par ses dates (émission, réception) et son statut (ex: "COMMANDÉE", "REÇUE"). L'association avec les produits se fait via l'entité **LigneCommande** (relation "Many-to-Many"), qui fige la quantité et le prix d'achat réel au moment de l'ordre. Enfin, le cycle d'achat intègre désormais une entité **Facture**. Celle-ci

est liée par une contrainte d'unicité stricte ("1-1") à la Commande, garantissant qu'une commande correspond à une et une seule facture enregistrée dans le système.

4. Gestion des Stocks Pour le pilotage des ressources, l'entité **Stock** maintient une relation "1-1" (un-pour-un) stricte avec l'entité Produit. Un produit possède une unique ligne de stock, ce qui centralise la gestion des quantités disponibles et des seuils d'alerte pour le réapprovisionnement automatique.

7.2 Modèle MLD :



- **Ville ↔ Fournisseur :** La relation est de type "1,n" : une ville peut héberger plusieurs fournisseurs, mais un fournisseur est situé dans une seule ville.
- **TypeProduit ↔ Produit :** La relation est de type "1,n" : un type regroupe plusieurs produits.
- **Produit ↔ Stock :** La relation est de type "1,1" : un produit possède une seule ligne de stock correspondante (gestion centralisée).
- **Commande ↔ LigneCommande :** La commande est composée de plusieurs lignes (1,n).
- **Commande ↔ Facture :** La relation est de type "1,1" : une commande génère une seule facture unique.
- **Utilisateur↔ Commande :** Un utilisateur peut passer plusieurs commandes (0,n).
- **Fournisseur↔Contact :** Un fournisseur peut avoir 1 ou plusieur contact.

7.2.1 MLD Textuel :

Ville = (**idville** INT, **nomville** VARCHAR(100));

Fournisseur = (**idfournisseur** INT, **nomfournisseur** VARCHAR(50), **adresse** VARCHAR(255), **complementAdresse** VARCHAR(100) , **complementAdresse** TEXT,**actif** BOOLEEN #idville, #idcontact);

Contact = (**idcontact** INT, **nomContact** VARCHAR(100), **emailcontact** VARCHAR(255), **telephonecontact** VARCHAR(50), **remarque** TEXT, #idfournisseur);

TypeProduit = (**idtype** INT, **libelletype** VARCHAR(50));

Utilisateur= (**idUser** INT, **nomUser** VARCHAR(50), **prenomUser** VARCHAR(50), **loginUser** VARCHAR(50), **emailUser** VARCHAR(50), **pswUser** VARCHAR(255),**actif** BOOLEEN ,**roleUser** CHAR(50) COLLATE USER,ADMIN,GUEST);

Commande = (**idCommande** INT, **dateCommande** DATETIME, **dateReception** DATETIME, **actif** BOOLEEN,**statutCommande** CHAR(20) COLLATE BROUILLON, COMMANDEE, RECUE, ANNULEE, #idfournisseur, #idUser);

Facture = (**idFacture** INT, **numeroFacture** VARCHAR(50), **dateFacture** DATETIME, **montantTotal** DECIMAL(10,2), **fichierPDF** VARCHAR(255), #idCommande);

Produit = (**idproduit** INT, **libelleproduit** VARCHAR(100), **prixunitaire** DECIMAL(10,2),**poids** DECIMAL(10,2),**description** TEXT,**actif** BOOLEEN, #idtype);

Stock = (**idStock** INT, **libelleStock** VARCHAR(50), **quantite** INT,**seuilAlerte** INT, #idproduit*);

Possede = (#idproduit, # idfournisseur, **refChezFournisseur** VARCHAR(50), **prixReference** DECIMAL(10,2) , **delaiLivraisonJours** INT);

ligneCommande = (#idproduit, #idCommande, **quantite** INT, **prixAchat** DECIMAL(10,2));

7.3 Modèle MPD :

```sql
/* ==================================================================
   BASE DE DONNEES :
   ================================================================== */

-- 1. Table Villes
CREATE TABLE villes (
    idville BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    nomville VARCHAR(100) NOT NULL,
    codePostal VARCHAR(10) NOT NULL,
    created_at TIMESTAMP NULL,
    updated_at TIMESTAMP NULL,
    UNIQUE KEY (nomville, codePostal)
);

-- 2. Table Type_Produits
CREATE TABLE type_produits (
    idtype BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    libelletype VARCHAR(50) NOT NULL UNIQUE,
    created_at TIMESTAMP NULL,
    updated_at TIMESTAMP NULL
);

-- 3. Table Utilisateurs
CREATE TABLE utilisateurs (
    idUser BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    nomUser VARCHAR(50) NOT NULL,
    prenomUser VARCHAR(50) NOT NULL,
    loginUser VARCHAR(50) NOT NULL,
    emailUser VARCHAR(50) NOT NULL UNIQUE,
    pswUser VARCHAR(255) NOT NULL,
    roleUser ENUM('USER', 'ADMIN', 'GUEST') NOT NULL DEFAULT 'GUEST',
    actif BOOLEAN NOT NULL DEFAULT TRUE,
    created_at TIMESTAMP NULL,
    updated_at TIMESTAMP NULL
);

-- 4. Table Fournisseurs
CREATE TABLE fournisseurs (
    idfournisseur BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    nomfournisseur VARCHAR(50) NOT NULL UNIQUE,
    adresse VARCHAR(255) NOT NULL,
    complementAdresse VARCHAR(100) NULL,
    siteWeb VARCHAR(255) NULL,
    details TEXT NULL,
```

```
    actif BOOLEAN NOT NULL DEFAULT TRUE,
    idville BIGINT UNSIGNED NOT NULL,
    created_at TIMESTAMP NULL,
    updated_at TIMESTAMP NULL,
    FOREIGN KEY (idville) REFERENCES villes(idville) ON DELETE RESTRICT ON
UPDATE CASCADE
);

-- 5. Table Contacts
CREATE TABLE contacts (
    idcontact BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    nomContact VARCHAR(100) NOT NULL,
    emailcontact VARCHAR(255) NOT NULL UNIQUE,
    telephonecontact VARCHAR(50) NULL,
    remarque TEXT NULL,
    idfournisseur BIGINT UNSIGNED NOT NULL,
    created_at TIMESTAMP NULL,
    updated_at TIMESTAMP NULL,
    FOREIGN KEY (idfournisseur) REFERENCES fournisseurs(idfournisseur) ON
DELETE CASCADE ON UPDATE CASCADE
);

-- 6. Table Produits
CREATE TABLE produits (
    idproduit BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    libelleproduit VARCHAR(100) NOT NULL UNIQUE,
    prixunitaire DECIMAL(10, 2) NULL,
    poidsEnKG DECIMAL(15, 2) NULL,
    description TEXT NULL,
    actif BOOLEAN NOT NULL DEFAULT TRUE,
    idtype BIGINT UNSIGNED NOT NULL,
    created_at TIMESTAMP NULL,
    updated_at TIMESTAMP NULL,
    FOREIGN KEY (idtype) REFERENCES type_produits(idtype) ON DELETE RESTRICT
ON UPDATE CASCADE
);

-- 7. Table Stocks
CREATE TABLE stocks (
    idStock BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    libelleStock VARCHAR(50) NULL,
    quantite INT NOT NULL DEFAULT 0,
    seuilAlerte INT NOT NULL DEFAULT 5,
    idproduit BIGINT UNSIGNED NOT NULL UNIQUE,
    created_at TIMESTAMP NULL,
    updated_at TIMESTAMP NULL,
```

```
    FOREIGN KEY (idproduit) REFERENCES produits(idproduit) ON DELETE CASCADE
ON UPDATE CASCADE
);

-- 8. Table Possedes (Table pivot Catalogue)
CREATE TABLE possedes (
    idproduit BIGINT UNSIGNED NOT NULL,
    idfournisseur BIGINT UNSIGNED NOT NULL,
    prixReference DECIMAL(10, 2) NULL,
    delaiLivraisonJours INT NULL,
    refProduitChezFournisseur VARCHAR(50) NULL,
    created_at TIMESTAMP NULL,
    updated_at TIMESTAMP NULL,
    PRIMARY KEY (idproduit, idfournisseur),
    FOREIGN KEY (idproduit) REFERENCES produits(idproduit) ON DELETE CASCADE
ON UPDATE CASCADE,
    FOREIGN KEY (idfournisseur) REFERENCES fournisseurs(idfournisseur) ON
DELETE CASCADE ON UPDATE CASCADE
);

-- 9. Table Commandes
CREATE TABLE commandes (
    idCommande BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    numeroCommande VARCHAR(50) NOT NULL,
    dateCommande DATETIME NOT NULL,
    dateReception DATETIME NULL,
    statutCommande ENUM('BROUILLON', 'COMMANDEE', 'RECUE', 'ANNULEE') NOT
NULL,
    details TEXT NULL,
    fichierPDF VARCHAR(255) NULL,
    actif BOOLEAN NOT NULL DEFAULT TRUE,
    idfournisseur BIGINT UNSIGNED NOT NULL,
    idUser BIGINT UNSIGNED NOT NULL,
    created_at TIMESTAMP NULL,
    updated_at TIMESTAMP NULL,
    FOREIGN KEY (idfournisseur) REFERENCES fournisseurs(idfournisseur) ON
DELETE RESTRICT ON UPDATE CASCADE,
    FOREIGN KEY (idUser) REFERENCES utilisateurs(idUser) ON DELETE RESTRICT
ON UPDATE CASCADE
);

-- 10. Table Factures
CREATE TABLE factures (
    idFacture BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    numeroFacture VARCHAR(50) NOT NULL,
    bonDeLivraison VARCHAR(50) NOT NULL,
```

```
    dateFacture DATE NOT NULL,
    montantTotal DECIMAL(10, 2) NULL,
    fichierPDF VARCHAR(255) NULL,
    idCommande BIGINT UNSIGNED NOT NULL UNIQUE,
    created_at TIMESTAMP NULL,
    updated_at TIMESTAMP NULL,
    FOREIGN KEY (idCommande) REFERENCES commandes(idCommande) ON DELETE
CASCADE ON UPDATE CASCADE
);
```

7.3.1 Description du MPD :

Gestion des Accès et des Tiers La table **Utilisateur** centralise les informations de connexion (mot de passe hashé) et les rôles des employés, avec une contrainte d'unicité sur l'email pour garantir l'intégrité des comptes. Concernant les partenaires externes, la structure a été affinée : la table **Fournisseur** représente l'entité morale (l'entreprise) et contient désormais son adresse physique complète liée à une **Ville**. Pour gérer finement la communication, une nouvelle table **Contact** a été introduite. Elle permet d'enregistrer plusieurs interlocuteurs (ex: commercial, comptable) pour un même fournisseur, chacun avec ses coordonnées propres (email, téléphone).

Catalogue et Gestion des Stocks Le catalogue est géré par la table **Produit**, catégorisée par **TypeProduit**. La fiche produit a été enrichie pour inclure le **poids** et une **description**, données essentielles pour la logistique. La particularité du modèle réside dans la table **Stock**, qui possède une contrainte **`UNIQUE(idproduit)`**. Cela garantit qu'il n'existe qu'une seule ligne de stock par produit, centralisant ainsi la gestion des quantités en temps réel et des seuils d'alerte.

Workflow d'Approvisionnement Le processus d'achat est matérialisé par la table **Commande**, qui lie un utilisateur (l'acheteur) à un fournisseur. Les détails précis (quantités, prix d'achat à l'instant T) sont stockés dans la table associative **LigneCommande**. Enfin, la gestion comptable est assurée par la table **Facture**. Celle-ci est liée à la commande par une contrainte d'unicité ("One-to-One"), assurant qu'une commande validée ne correspond qu'à une seule facture enregistrée dans le système.

7.4 Automatisation (Triggers SQL)

```
/* ==================================================================
   TRIGGERS (AUTOMATISATION)
   ================================================================== */

-- 1. Initialisation automatique du stock
-- Règle : Un produit créé doit immédiatement avoir une fiche de stock à
0.
CREATE TRIGGER InitStockAfterProductCreation
```

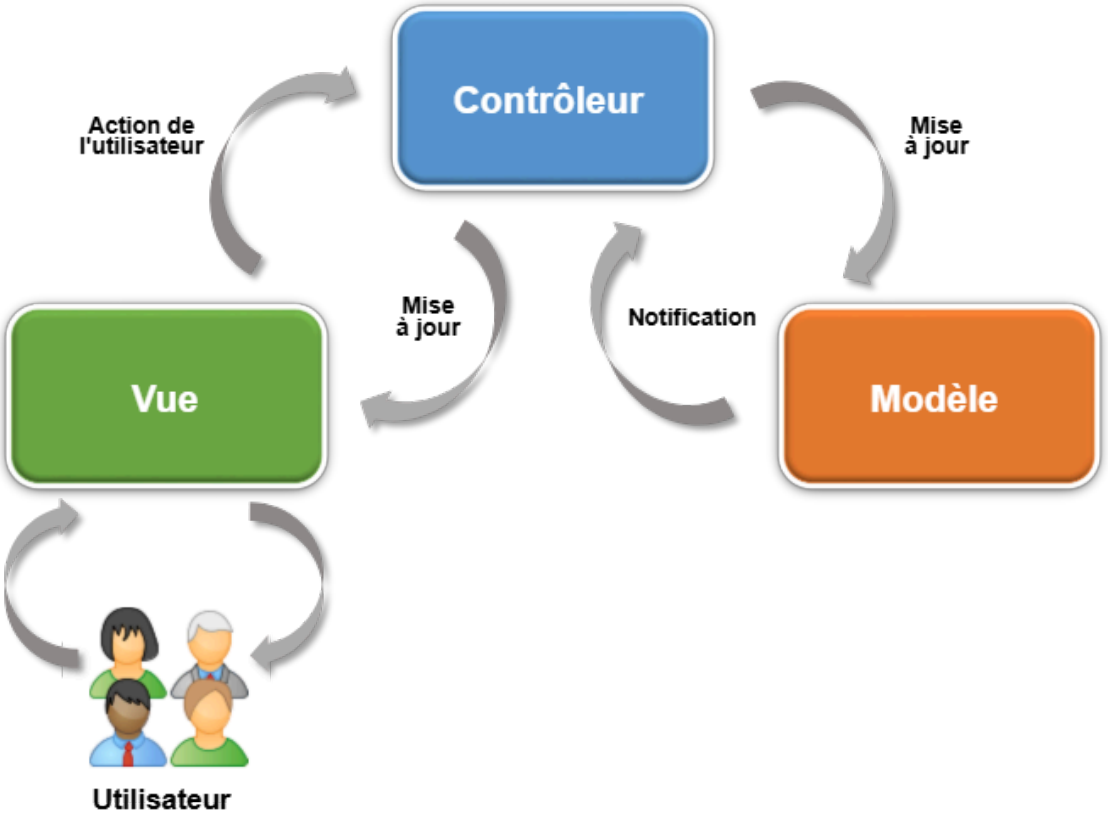
```sql
AFTER INSERT ON Produit
BEGIN
    INSERT INTO Stock (libelleStock, quantite, seuilAlerte, idproduit)
    VALUES ('Stock ' || NEW.libelleproduit, 0, 5, NEW.idproduit);
END;

-- 2. Mise à jour du stock à la réception
-- Règle : Quand le statut passe à "RECUE", on incrémente le stock.
CREATE TRIGGER UpdateStockAfterReception
AFTER UPDATE OF statutCommande ON Commande
FOR EACH ROW
WHEN NEW.statutCommande = 'RECUE' AND OLD.statutCommande != 'RECUE'
BEGIN
    UPDATE Stock
    SET quantite = quantite + (
        SELECT quantite
        FROM ligneCommande
        WHERE ligneCommande.idproduit = Stock.idproduit
        AND ligneCommande.idCommande = NEW.idCommande
    )
    WHERE idproduit IN (
        SELECT idproduit
        FROM ligneCommande
        WHERE idCommande = NEW.idCommande
    );
END;
```

Automatisation des flux de données Afin de garantir l'intégrité des données et de réduire les tâches manuelles, deux déclencheurs (Triggers) ont été implémentés directement dans la base de données SQLite :

1. **`InitStockAfterProductCreation`** : Ce trigger assure la cohérence de la relation "1-1" entre Produit et Stock. Dès qu'un produit est ajouté au catalogue, sa ligne de stock correspondante est créée automatiquement avec une quantité initiale nulle et un seuil d'alerte par défaut.
2. **`UpdateStockAfterReception`** : Conformément au workflow d'approvisionnement, ce trigger surveille le changement de statut d'une commande. Lorsque celle-ci passe à l'état **"RECUE"**, le système calcule automatiquement les quantités livrées pour chaque article et met à jour les niveaux de stock correspondants en temps réel, sans intervention de l'utilisateur.

8. Modèle MVC

MVC (Modèle-Vue-Contrôleur) est un modèle de conception logicielle couramment utilisé pour implémenter les interfaces utilisateurs, les données et la logique de contrôle. Le framework **Laravel**, choisi pour le backend du projet (voir 3.2), est construit sur cette architecture. Elle met l'accent sur la séparation entre la logique métier du logiciel et l'affichage. Cette « séparation des responsabilités » permet une meilleure répartition du travail, une meilleure organisation du code et une maintenance facilitée. D'autres modèles de conception sont basés sur MVC, comme MVVM (Modèle-Vue-VueModèle), MVP (Modèle-Vue-Présentateur) et MVW (Modèle-Vue-Whatever).

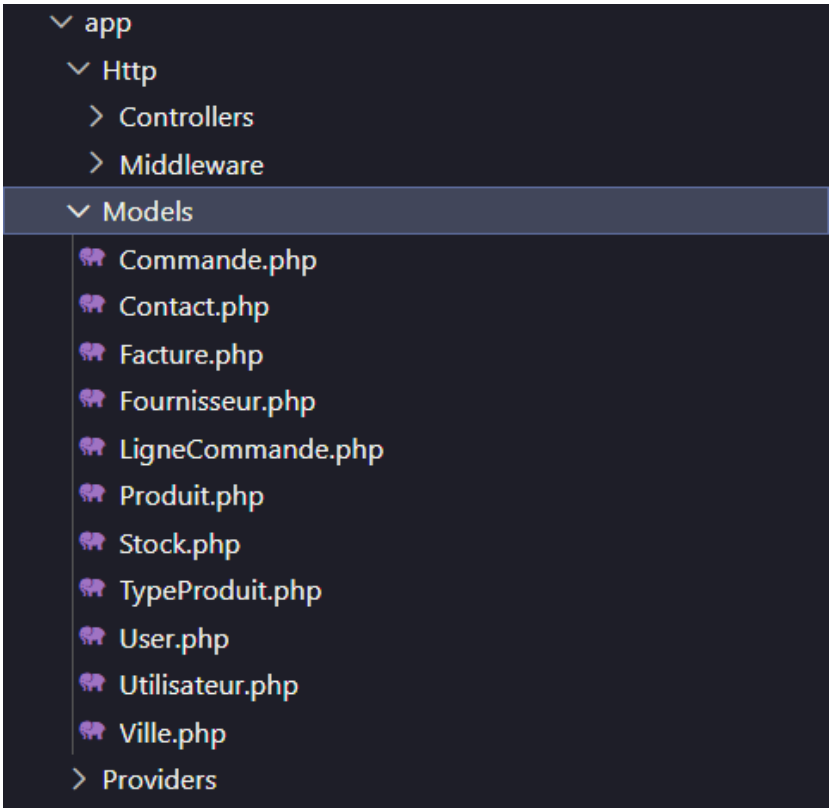


8.1 Modèle (Model)

Le **Modèle** représente la structure des données et la logique métier de l'application. Dans l'architecture Laravel de LoDaStock, chaque table de la base de données SQLite (décrite dans la section 7) possède une classe correspondante dans le dossier `app/Models`.

Ces modèles utilisent **Eloquent ORM** (Object-Relational Mapping), ce qui permet d'interagir avec la base de données en manipulant des objets PHP plutôt qu'en écrivant des requêtes SQL complexes.

- **Rôle principal :** Assurer la communication avec la base de données (récupération, insertion, modification, suppression) et définir les relations entre les tables.
- Exemples de modèles implémentés :
  - **`Produit.php`** : Représente un article du catalogue. Il contient la logique liant un produit à son `TypeProduit` (relation *BelongsTo*) et à son unique `Stock` (relation *HasOne*).
  - **`Commande.php`** : Gère les informations des commandes. Elle définit les relations avec le `Fournisseur` et les `LigneCommande` (les articles contenus dans la commande).
  - **`Utilisateur.php`** : Gère l'authentification et les droits d'accès des employés (rôles Admin ou User ou Guest).
  - **`Fournisseur.php`** et **`Contact.php`** : Structurent les informations des partenaires et de leurs interlocuteurs respectifs.

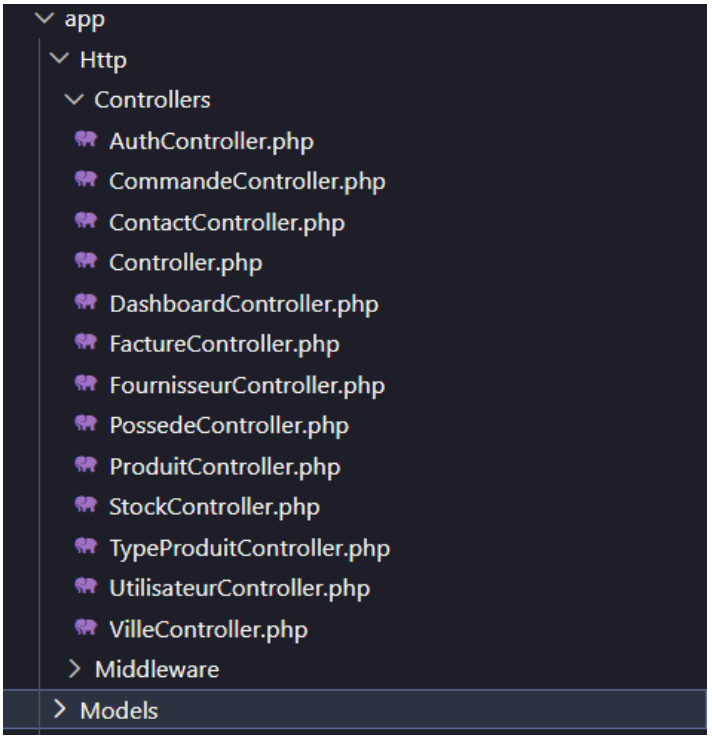


8.2 Contrôleur (Controller)

Le **Contrôleur** agit comme le "chef d'orchestre" de l'application. Il reçoit les requêtes HTTP envoyées par l'utilisateur (clic sur un lien, soumission d'un formulaire), traite ces demandes en faisant appel aux **Modèles**, et renvoie la réponse appropriée (généralement une **Vue**).

Les contrôleurs sont situés dans le dossier `app/Http/Controllers` et regroupent la logique par module fonctionnel.

- **Fonctionnement type (CRUD) :** La plupart des contrôleurs de LoDaStock suivent la structure standard des ressources Laravel :
  - `index()` : Affiche la liste des éléments (ex: liste des produits).
  - `create()` / `store()` : Affiche le formulaire de création et enregistre le nouvel élément.
  - `show()` : Affiche les détails d'un élément spécifique.
  - `edit()` / `update()` : Gère la modification d'un élément.
  - `destroy()` : Supprime un élément.
- Contrôleurs clés du projet :
  - **`ProduitController.php`** : Gère l'administration du catalogue (ajout, modification de produits).
  - **`CommandeController.php`** : Pilote le workflow d'approvisionnement. C'est ici que sont gérés les changements d'états (ex: passage de "COMMANDÉE" à "REÇUE") qui déclenchent les triggers de mise à jour de stock.
  - **`DashboardController.php`** : Agrège les données critiques (alertes de stock, commandes en attente) pour les présenter sur l'écran d'accueil.
  - **`AuthController.php`** : Sécurise l'accès en gérant la connexion et la déconnexion des utilisateurs.

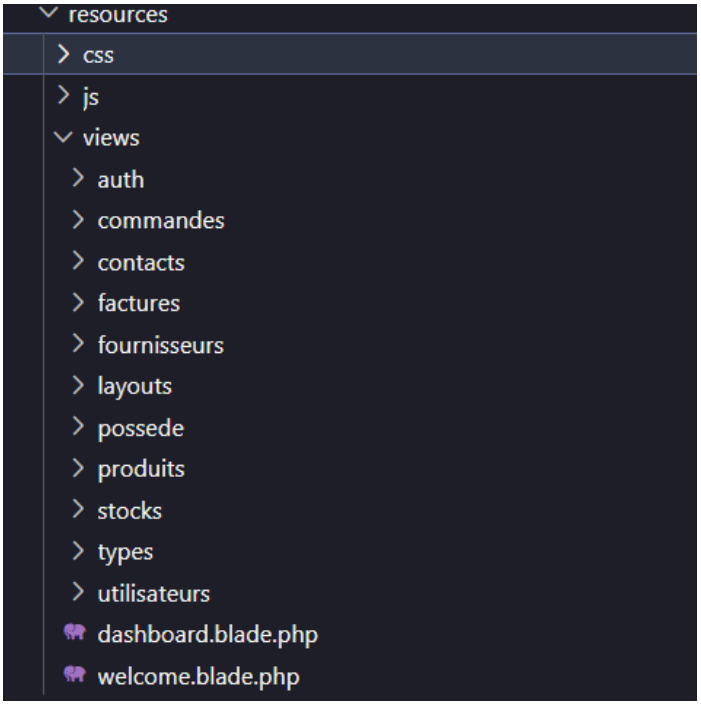


8.3 Vue (View)

La **Vue** correspond à l'interface graphique présentée à l'utilisateur (le "Frontend"). LoDaStock utilise le moteur de template **Blade** fourni par Laravel, reconnaissable à l'extension `.blade.php` des fichiers situés dans `resources/views`.

Blade permet d'intégrer dynamiquement les données transmises par le Contrôleur au sein du code HTML, tout en simplifiant la syntaxe (boucles, conditions).

- Structure et Organisation :
  - **Dossiers modulaires :** Les vues sont organisées dans des sous-dossiers correspondant aux fonctionnalités (ex: `resources/views/produits/`, `resources/views/commandes/`, `resources/views/fournisseurs/`).
  - **Layouts (`layouts/`)** : Le fichier principal `app.blade.php` (ou `main.blade.php`) définit le squelette unique de l'application (en-tête, barre de navigation latérale, pieds de page). Toutes les autres vues "héritent" de ce modèle, garantissant une cohérence visuelle sur toutes les pages.
  - **Composants :** L'interface utilise le framework CSS **Bootstrap 5** pour le design responsive et **Alpine.js** pour les interactions dynamiques (comme les fenêtres modales de confirmation) sans rechargement de page.



9. Problème rencontré

9.1 Gestion des envois d'emails et Commandes Fournisseurs

Problème rencontré : Limitations du protocole SMTP Lors du développement, une contrainte technique est apparue concernant l'envoi automatisé d'emails via le protocole SMTP. Pour que l'application puisse envoyer des emails automatiquement au nom de différents utilisateurs connectés, il aurait fallu configurer une adresse d'expédition unique et partagée (ex: *no-reply@lodastock.com*). Or, dans le contexte actuel, chaque employé doit pouvoir envoyer des commandes en son nom propre, et l'infrastructure ne permettait pas de configurer dynamiquement le serveur SMTP pour chaque utilisateur individuel.

Solutions mises en place :

Face à cette contrainte et pour s'adapter aux processus réels de l'entreprise, le fonctionnement a été modifié de deux manières :

1. **Génération automatique, envoi manuel (Mailto) :** L'application ne s'occupe plus de l'envoi direct (qui risquerait d'être bloqué ou de partir d'une mauvaise adresse). À la place, elle **génère automatiquement** le contenu du mail (destinataire, objet, corps du message avec les détails de la commande) et ouvre le logiciel de messagerie par défaut de l'utilisateur (Outlook, Thunderbird, etc.).
    - *Avantage :* L'utilisateur garde le contrôle, peut relire le message, ajouter des précisions si nécessaire, et l'email part bien de son adresse personnelle.
2. **Redirection vers les portails fournisseurs :** L'analyse des processus a révélé que la commande par email n'était pas systématique. De nombreux fournisseurs disposent de leur propre site e-commerce B2B.
    - *Évolution :* Un champ **"Site Web"** a été ajouté à la table des fournisseurs.
    - *Fonctionnalité :* Lors d'une commande, l'application propose désormais un lien de redirection directe vers le site du fournisseur concerné, permettant à l'utilisateur de passer commande via l'interface web du tiers plutôt que par email.

9.2 Intégrité des données et politique de suppression

Problème rencontré : Perte de traçabilité La suppression définitive ("suppression physique") d'enregistrements dans la base de données pose un risque majeur pour la cohérence de l'historique. Si un utilisateur supprime un produit ou un fournisseur de la base, toutes les anciennes commandes liées à ces entités deviendraient orphelines ou incomplètes. Il serait alors impossible de retracer l'origine d'un ancien stock ou de consulter l'historique des achats d'un produit disparu, ce qui est contraire aux règles de gestion comptable et logistique.

Solution mise en place : Le système de désactivation

Pour garantir la pérennité des données tout en permettant de nettoyer les listes d'éléments obsolètes, une distinction stricte des droits a été établie :

1. **Pour les utilisateurs standards (Désactivation) :** L'utilisateur ne possède pas le droit de supprimer définitivement une donnée critique (Fournisseur, Produit, Commande validée). À la place, il dispose d'une fonction de **désactivation**.
    - *Fonctionnement technique :* Un champ (booléen) dans la base de données passe de "Actif" à "Inactif".
    - *Résultat :* L'élément n'apparaît plus dans les listes de sélection pour les nouvelles commandes (ce qui évite les erreurs), mais il reste présent dans la base de données pour assurer l'affichage correct des archives et des statistiques passées.

2. **Pour l'Administrateur (Suppression définitive) :** Seul le profil **Admin** conserve le privilège de suppression définitive. Ce droit est réservé à des cas particuliers, principalement pour la maintenance de la base de données ou la correction immédiate d'erreurs de saisie (ex: création d'un doublon par erreur), et doit être utilisé avec précaution.

9.3 Système d'alerte de stock : Alternative aux notifications par email

Problème rencontré : Impossibilité d'envoyer des alertes automatisées Initialement, le cahier des charges prévoyait que l'application envoie automatiquement un email aux utilisateurs dès qu'un produit atteignait son seuil d'alerte de stock. Cependant, en raison des mêmes contraintes de configuration du serveur SMTP évoquées précédemment (impossibilité d'utiliser une adresse d'envoi automatisée partagée), ce processus de notification en arrière-plan n'était pas réalisable. Il était pourtant crucial de garantir que les utilisateurs soient informés à temps des nécessités de réapprovisionnement.

Solution mise en place : Modale d'alerte proactive à la connexion Pour pallier l'absence d'emails automatiques, le système d'alerte a été repensé et intégré directement au cœur de l'interface utilisateur. Désormais, une fenêtre modale (pop-up) s'affiche automatiquement au lancement de l'application ou à l'arrivée sur le tableau de bord, alertant l'utilisateur de manière proactive.

Afin d'optimiser l'expérience utilisateur et de rendre cette alerte efficace sans être bloquante, plusieurs fonctionnalités ont été développées :

- **Affichage récapitulatif :** La modale génère dynamiquement la liste de tous les produits dont la quantité actuelle est inférieure ou égale à leur seuil d'alerte.
- **Ergonomie visuelle :** La fenêtre utilise un code couleur spécifique (en-tête rouge) pour capter immédiatement l'attention de l'utilisateur sur la criticité de la situation.
- **Action immédiate (Call-to-Action) :** Un bouton intégré permet de rediriger l'utilisateur directement vers la page de gestion des stocks, avec un filtre pré-appliqué n'affichant que les articles en alerte. Cela facilite et accélère le processus de réapprovisionnement.
- **Gestion de la récurrence (LocalStorage) :** Pour éviter que la modale ne devienne une nuisance (par exemple, en s'affichant à chaque retour sur l'accueil), un système de temporisation a été mis en place. L'utilisateur dispose d'une option *"Ne plus afficher aujourd'hui"*. Ce choix est sauvegardé localement dans le navigateur web (*localStorage*), garantissant que l'alerte ne s'affichera qu'une seule fois par jour, sauf si l'utilisateur décide de la réactiver.

10. Gestion des erreurs

Pour garantir une application robuste et fluide, une architecture complète de gestion des erreurs a été mise en place. L'objectif était d'éviter que l'utilisateur ne se retrouve face à des pages d'erreurs techniques ou ne comprenne pas pourquoi son action a échoué. Cette gestion opère à deux niveaux : le serveur (Backend) et l'interface (Frontend).

10.1 Côté Serveur (Backend) : Centralisation et Sécurité de la donnée

La gestion des erreurs côté serveur repose sur plusieurs mécanismes standardisés dans Laravel :

- **Gestionnaire d'exceptions global :** L'application est configurée (bootstrap/app.php) pour intercepter de manière transparente les exceptions courantes telles que les ressources introuvables (Erreur 404), les accès refusés (Erreur 403) ou l'expiration de session (Erreur 419). Au lieu de faire planter l'application, le système renvoie l'utilisateur vers des vues d'erreur personnalisées ou génère des réponses JSON propres si l'action était exécutée en arrière-plan.
- **Standardisation via le Trait HandlesErrors :** Un composant spécifique a été développé pour unifier la manière dont les contrôleurs répondent (succès, avertissement, erreur). Ce composant intègre notamment des mécanismes de protection de la base de données (tryOrFail, canDelete). Par exemple, si un utilisateur tente de supprimer un fournisseur qui est déjà lié à des commandes existantes, le système intercepte l'erreur de contrainte d'intégrité SQL et renvoie un avertissement clair au lieu de générer une erreur critique.
- **Traduction et clarté des validations :** Tous les messages de validation des formulaires ont été entièrement traduits et adaptés en français (lang/fr/validation.php). Les noms des champs techniques en base de données sont remplacés par des termes métier compréhensibles par l'employé (par exemple, afficher "Le champ nom du produit est obligatoire" plutôt que "Le champ libelleproduit est requis").

10.2 Côté Client (Frontend) : Retours visuels dynamiques

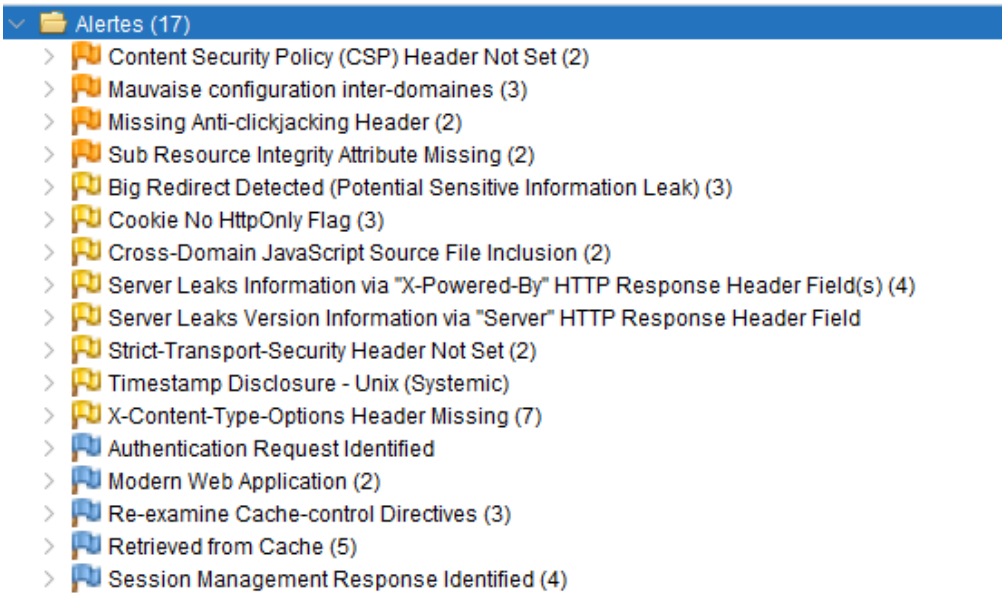
Pour les actions effectuées sans rechargement de page (requêtes AJAX), un module JavaScript personnalisé (error-handler.js) a été conçu pour interagir directement avec l'utilisateur :

- **Système de notifications "Toast" :** Les fenêtres d'alertes traditionnelles du navigateur ont été remplacées par des notifications non-bloquantes (Toasts). Elles s'affichent dynamiquement en haut de l'écran avec un code couleur précis (vert pour le succès, rouge pour l'erreur, jaune pour l'avertissement) et une barre de progression temporelle. Elles informent l'utilisateur du résultat de son action (ex: "Commande enregistrée avec succès") puis disparaissent automatiquement sans gêner sa navigation.
- **Intercepteur AJAX intelligent :** Ce module intercepte silencieusement toutes les communications avec le serveur. S'il détecte une erreur de saisie (code 422), il extrait les messages d'erreur de Laravel et les affiche dans un Toast. S'il détecte que la session de l'utilisateur a expiré pendant son absence (code 401 ou 419), il le notifie et prépare un rechargement de la page pour le rediriger vers l'écran de connexion en toute sécurité.

11. Sécurité et Audit des Vulnérabilités (OWASP ZAP)

Afin de garantir un haut niveau de sécurité pour l'application LoDaStock et de protéger les données de l'entreprise, un audit de sécurité automatisé a été réalisé à l'aide de l'outil **OWASP ZAP (Zed Attack Proxy)**. Cet audit a permis d'identifier et de corriger plusieurs vulnérabilités potentielles, principalement liées à la configuration du serveur et aux échanges HTTP.

Les corrections ont été centralisées via la création d'un Middleware dédié (`SecurityHeaders.php`) et la modification des fichiers de configuration (`session.php`, `.htaccess`)



11.1 Renforcement des politiques de sécurité (Mitigation)

Afin de corriger plusieurs vulnérabilités de niveau modéré à faible (identifiées par ZAP), des en-têtes de sécurité HTTP stricts ont été implémentés via un *middleware* dédié.

Domaine de Sécurité	Alerte ZAP Originale	Mesures Correctives Détaillées
Protection Contre l'Intégration (Clickjacking / Cross-Domain)	`Missing Anti-clickjacking Header` / `Cross-Domain`	Ajout des en-têtes `X-Frame-Options: SAMEORIGIN` et de la directive `frame-ancestors 'self'` dans la CSP. Configuration de `Referrer-Policy:`

	`misconfiguration`	`strict-origin-when-cross-origin`.
Prévention XSS et Usurpation MIME	`X-Content-Type-Options Missing` / `XSS Protection` / `CSP Header Not Set`	Déploiement d'une **Content-Security-Policy (CSP)** stricte autorisant spécifiquement les ressources nécessaires (ex: API Adresse du gouvernement via `connect-src`). Ajout des en-têtes `X-Content-Type-Options: nosniff` et `X-XSS-Protection: 1; mode=block`.
Gestion du Cache et HSTS	`Re-examine Cache-Control` / `Strict-Transport-Security Not Set`	Application des directives `Cache-Control: no-store, no-cache, must-revalidate, private` pour empêcher la mise en cache des données sensibles de session. **Note sur HSTS :** L'en-tête `Strict-Transport-Security` a été commenté pour maintenir l'accessibilité en développement local (HTTP) et faciliter la démonstration.
Intégrité des Ressources Externes (SRI)	`Sub Resource Integrity Missing` / `Cross-Domain JS Source File`	Ajout d'une empreinte cryptographique (SRI SHA-384) sur le script externe Alpine.js (v3.14.8) dans `main.blade.php`, assurant l'intégrité du code téléchargé depuis le CDN.
Sécurisation des Sessions/Cookies	`Cookie No HttpOnly Flag`	Configuration stricte dans `config/session.php` avec `http_only: true` (inaccessible via JavaScript), `encrypt: true`, et `same_site: 'strict'` (protection contre les attaques CSRF).
Anonymisation du Serveur (Information Disclosure)	`Server Leaks X-Powered-By`	Suppression des en-têtes d'information (`X-Powered-By` et `Server`) dans les réponses HTTP afin de masquer la *stack* technique sous-jacente aux attaquants potentiels.

11.2 Alertes informatives et Faux positifs

L'audit a également remonté des alertes purement informatives qui ne constituent pas des failles et ne requièrent aucune modification du code :

- **Authentication Request Identified :** Comportement normal du système d'authentification natif de Laravel.
- **Modern Web Application :** ZAP identifie l'utilisation de frameworks modernes (Alpine.js, Tailwind), ce qui est un constat technique et non une vulnérabilité.
- **Timestamp Disclosure :** Les horodatages présents dans les requêtes/logs ne sont pas exposés de manière critique côté client.

12. Conteneurisation et Déploiement

Afin d'optimiser et de standardiser les processus d'installation, de maintenance et de mise en production de l'application LoDaStock, une architecture conteneurisée reposant sur Docker a été privilégiée. Au lieu d'installer manuellement l'ensemble des dépendances (environnement PHP, serveur web, utilitaires de compilation) sur l'infrastructure physique de l'entreprise, l'application et son contexte d'exécution ont été encapsulés au sein d'environnements isolés.

12.1 Les avantages de la solution Docker

L'adoption de Docker confère plusieurs avantages stratégiques et techniques majeurs au projet :

- **Portabilité et homogénéité :** L'application s'exécute dans un environnement strictement standardisé. Cela élimine les risques de régressions liés aux différences de configuration entre les postes de développement et les serveurs de production.
- **Déploiement automatisé :** Grâce à l'utilisation d'un fichier d'orchestration `docker-compose.yml`, l'initialisation de l'infrastructure applicative s'effectue via une commande unique, s'affranchissant ainsi des configurations manuelles complexes sur la machine hôte.
- **Isolation des processus :** L'application s'exécute dans un conteneur dédié et hermétique, garantissant l'absence d'interférences ou de conflits avec d'autres services potentiellement hébergés sur le même serveur.
- **Sécurité et maintenabilité :** Les opérations de maintenance (telles qu'une montée de version de PHP ou d'un composant serveur) s'effectuent par le simple déploiement d'une nouvelle image Docker, sans risquer d'altérer la stabilité du système d'exploitation hôte.

12.2 Architecture mise en place

L'orchestration du déploiement est assurée par l'outil Docker Compose. Dans le cadre de LoDaStock, l'environnement de production s'articule autour des éléments suivants :

- **Conteneur Applicatif / Web (`rayaneww/lodastock`) :** Le service principal s'appuie sur l'image Docker personnalisée `rayaneww/lodastock`. Cette image embarque de manière autonome le code source de l'application, le serveur web, ainsi que la version exacte de PHP et les extensions requises par le framework Laravel.
- **Stratégie de persistance (Volumes) :** La nature éphémère des conteneurs impose une gestion rigoureuse des données. Des volumes Docker sont spécifiquement configurés pour extraire et sauvegarder la base de données SQLite ainsi que les fichiers importés par les utilisateurs (uploads). Ce mécanisme garantit la pérennité des données de l'entreprise, même lors de la destruction ou de la mise à jour des conteneurs.

Cette approche architecturale moderne dote LoDaStock d'une infrastructure résiliente, facilitant grandement la supervision par les équipes techniques de Loiselet & Daigremont tout en anticipant les futurs besoins d'évolution.

13. Axes d'amélioration et Évolutions futures

Bien que l'application réponde aux objectifs initiaux du cahier des charges, plusieurs évolutions techniques et fonctionnelles peuvent être envisagées pour accompagner la croissance des besoins de l'entreprise et optimiser davantage les processus.

- **Génération automatisée de documents (PDF) et Archivage :**
  Actuellement, le système gère les statuts et permet de lier des fichiers existants. L'intégration d'une librairie tierce (comme DomPDF) permettrait de générer nativement et automatiquement les bons de commande et les factures au format PDF lors de leur validation. Cela renforcerait l'historique, la traçabilité et faciliterait l'archivage légal.
- **Migration vers un Serveur de Base de Données (SGBD) dédié :**
  Le choix de SQLite est parfaitement adapté au périmètre et à la volumétrie actuels. Cependant, si le nombre d'utilisateurs simultanés venait à augmenter (accès concurrents fréquents) ou si le volume de données devenait massif, il serait judicieux de migrer vers un moteur de base de données client-serveur plus robuste, tel que MySQL ou PostgreSQL. L'utilisation de l'ORM Eloquent de Laravel rendrait cette transition techniquement très souple.
- **Déploiement d'un service SMTP dédié :** Afin de contourner les limitations techniques rencontrées lors du développement (décrites dans les sections 9.1 et 9.3), la mise en place d'un relais SMTP propre à l'application ou l'utilisation d'une API transactionnelle (type SendGrid ou Mailgun) permettrait de réactiver l'envoi transparent des commandes aux fournisseurs et l'expédition d'alertes de stock automatisées par email en arrière-plan.
- **Export des données et Interfaçage Comptable :**
  Pour faciliter le travail des services administratifs et financiers, le développement de modules d'exportation (aux formats CSV ou Excel) serait un atout majeur. Cela

permettrait d'extraire rapidement la valorisation du stock en temps réel ou le bilan mensuel des achats par fournisseur pour les intégrer dans un logiciel comptable externe.

- **Intégration de la lecture de Codes-barres / QR Codes :**
  Pour accélérer les inventaires physiques et fiabiliser la réception des commandes, l'interface pourrait évoluer pour prendre en charge le scan de codes-barres ou de QR codes (via un terminal mobile ou une douchette USB). Cela réduirait drastiquement les erreurs de saisie manuelle lors des mouvements de stock.